



BINV1053

3.1 - Contrôleur et CRUD complet

Jérôme Plumat

Haute École Léonard de Vinci

2026





- 1 Introduction
- 2 Rappels
- 3 Mapper et DTO
- 4 CRUD et READ
- 5 Les autres opérations et leur happy path
- 6 Conclusion



1. Introduction



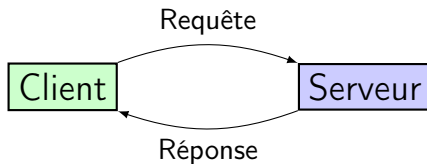
Objectifs

Dans cette séance, nous allons :

- Comprendre les Happy Path pour toutes les opérations de base du CRUD



2. Rappels





La requête est composée de plusieurs éléments, dont notamment :

- une **méthode http** : `GET`, `POST`, `PUT`, `DELETE` ;
- une **URL**, composée de plusieurs parties dont le nom de **ressource**, de potentiels **paramètres** et une **query** ;
- le corps (body) de la requête ;



Méthodes HTTP

Chacune des méthodes HTTP peut être utilisée pour effectuer une opération CRUD sur une ressource.

- GET : Read, paramètres dans l'URL ;
- POST : Create, données dans le body ;
- PUT : Update, données dans le body ;
- DELETE : Delete, paramètres dans l'URL ;

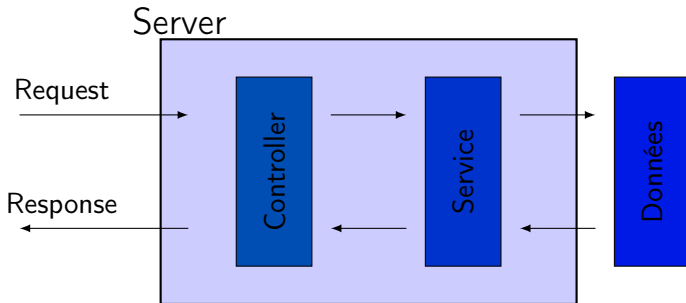


La réponse est composée de plusieurs éléments, dont :

- le corps de la réponse contenant du texte ou des données ;
- le statut indiquant la réussite ou l'échec de la requête, les principaux statuts de réponse sont :
 - ▶ 200 : OK ;
 - ▶ 201 : Created ;
 - ▶ 400 : Bad Request ;
 - ▶ 404 : Not Found ;
 - ▶ 500 : Internal Server Error. (semaine 4)

Server REST

Modèle du serveur



Notre serveur sera composé de plusieurs éléments que nous définirons dans les semaines à venir. Cette semaine (comme la semaine passée), nous nous concentrerons sur le contrôleur.



Le contrôleur réceptionne et renvoie

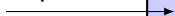
Le premier rôle du contrôleur est de réceptionner la requête et de renvoyer une réponse. Il est chargé de construire la réponse avec les statuts et les informations nécessaires.

Le contrôleur contrôle

Le second rôle du contrôleur est de contrôler les données. Il vérifie que les données de la requête sont correctes. Il vérifie que les données demandées sont bien présentes ou se sont créées avec succès.



Requête



1. Valider la requête
2. Appeler les services
3. Vérifier les réponses des services
- ▼ 4. Préparer et envoyer la réponse

Réponse





3. Mapper et DTO



- Dans le cadre de ce cours, un **DTO** (Data Transfer Object) est un objet qui transporte des données entre le serveur et un acteur extérieur (client, autre service, etc.).
- Il est utilisé pour encapsuler les données et les transmettre d'une couche à une autre.
- Les DTO sont souvent utilisés dans les APIs pour définir la structure des données échangées.



Mutliples représentations d'une même ressource

Dans le cadre d'un serveur, il est important de distinguer les différentes représentations d'une même ressource. A ce stade, nous avons deux représentations d'une même ressource :

- Un modèle : une représentation d'une ressource qui vit au sein de notre serveur.
- Un DTO (Data Transfer Object) : une ressource qui provient du client ou envoyée à celui-ci.



Mutiples représentations d'une même ressource

Dans la pratique, nous définissons différentes interfaces pour représenter ces deux types de ressources. Par exemple, pour la ressource Doctor, nous avons une interface Doctor et une interface DoctorDTO.

Par facilité, toutes les représentations seront placées dans le même fichier (`doctor.model.ts`).

Pourquoi utiliser différentes représentations ?

- Sécurité : les DTO peuvent masquer des informations sensibles du modèle ;
- Flexibilité : les DTO peuvent être adaptés à différents besoins (ex : `PatientShortDTO`, `PatientDTO`, etc.) ;
- Maintenance : les modèles peuvent évoluer indépendamment des DTO.



Exemples sur la ressource Doctor

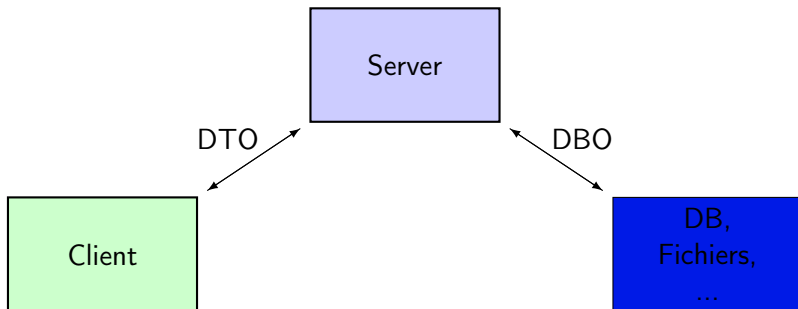
```

1  // file doctor.model.ts
2  export interface Doctor {
3    id: number;
4    firstName: string;
5    lastName: string;
6    specialty : string;
7  }
8
9  export interface DoctorShortDTO {
10    id: number;
11    firstName: string;
12    lastName: string;
13  }
14
15  export interface DoctorDTO {
16    id: number;
17    firstName: string;
18    lastName: string;
19    specialty : string;
20  }
    
```

Toutes les représentations d'une ressource



En plus du modèle et des DTO, nous avons aussi une représentation supplémentaire qui est la base de données (DBO - Database Object). Cette représentation est utilisée pour stocker les données dans la base de données et est généralement différente des autres représentations.





Un mapper est une classe qui permet de transformer des objets entre différents formats. Généralement, ces différents formats sont des représentations différentes d'une même ressource.

Exemple : on souhaite transformer un `Doctor` en `DoctorBD0` pour enregistrer les données de la ressource.

Un mapper est donc une boîte à outils qui centralise toute la logique de transformation entre différents formats.

Nous aurons un mapper pour chaque ressource. Par exemple, nous avons un mapper `DoctorMapper` et nous appelons la méthode `toBD0` pour transformer un `Doctor` en `DoctorBD0`.

Mapper

Example



```
1  export class DoctorMapper {  
2      public static toBDO(doctor: Doctor): DoctorBDO {  
3          return {  
4              id: doctor.id,  
5              first_name: doctor.firstName,  
6              last_name: doctor.lastName,  
7              speciality : doctor.speciality  
8          };  
9      }  
10  
11     public static fromDBO(dbo: DoctorBDO): Doctor {  
12         return {  
13             id: dbo.id,  
14             firstName: dbo.first_name,  
15             lastName: dbo.last_name,  
16             speciality : dbo.speciality  
17         };  
18     }  
19 }
```



On utilisera les mappers dans nos **services** pour transformer les models en DBOs et vice versa.

```
1  const doctorBDO: DoctorDBO = DoctorMapper.toBDO(doctor);  
2  // ...  
3  const doctor: Doctor = DoctorMapper.fromDBO(doctorBDO);
```

On utilisera également les mappers dans nos **contrôleurs** pour transformer les DTO en models et vice versa.

```
1  const doctor: Doctor = DoctorMapper.fromDTO(doctorDTO);  
2  // ...  
3  const doctorDTO: DoctorDTO = DoctorMapper.toDTO(doctor);
```



Avantages

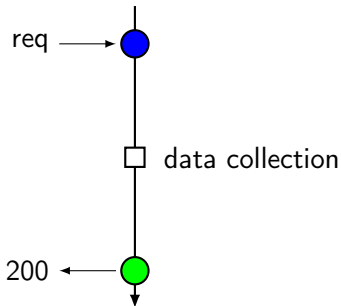
- Centralise la logique de transformation entre différents formats.
 - ▶ Lorsqu'on rajoute une propriété dans un modèle, la quantité de code à modifier est limitée.
- Permet de "cacher" la logique de transformation.
- Evite de dupliquer du code.



4. CRUD et READ



Objectif : récupérer tous les docteurs (sans filtrage).
exemple : GET `http://localhost:3000/doctors/`



- **Validation** : aucune validation nécessaire ;
- **Données** : tous les docteurs ;
- **Erreur** : aucune erreur possible à ce stade.



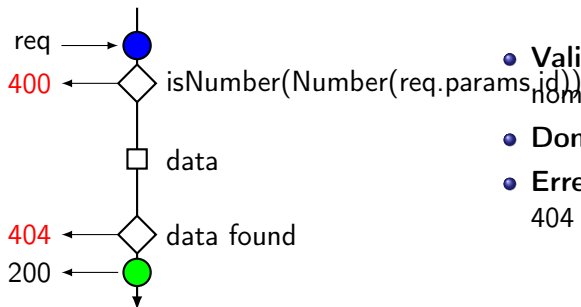
Le code correspondant est le suivant :

```
1 doctorsController .get("/", (req: Request, res: Response) => {  
2   const doctorsDB : Doctor[] = doctors; // static data, temporary solution  
3   const doctorsDTO: DoctorDTO[] = [];  
4   for(const doc of doctorsDB) {  
5     doctorsDTO.push(DoctorsMapper.toDTO(doc));  
6   }  
7   res.status(200).send(doctorsDTO);  
8 });
```



Objectif : récupérer un docteur sur base de son **id**.

exemple : GET `http://localhost:3000/doctors/1`



- **Validation** : id doit être un nombre ;
- **Données** : le docteur ;
- **Erreur** : 400 si id est incorrect, 404 si le docteur n'existe pas.



```
1 import { isNumber } from '../utils/guards';
2 doctorsController.get("/:id", (req: Request, res: Response) => {
3   const id = Number(req.params.id); // get the id from the URL
4   if (!isNumber(id)) {
5     res.status(400).send("Invalid id");
6     return;
7   }
8   let doctorIndex = -1;
9   for (let i = 0; i < doctors.length; i++) {
10     if (doctors[i].id === id) {
11       doctorIndex = i;
12       break;
13     }
14   }
15   if (doctorIndex === -1) {
16     res.status(404).send("Doctor not found");
17     return;
18   }
19   res.status(200).send(DoctorsMapper.toDTO(doctors[doctorIndex]));
20 });
```

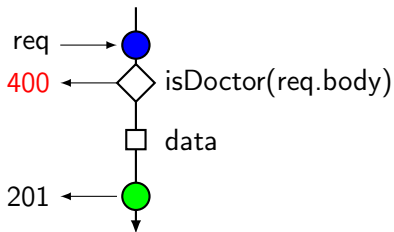


5. Les autres opérations et leur happy path



Objectif : créer un docteur sur base d'information.

exemple : POST `http://localhost:3000/doctors/`



- **Validation** : le body doit contenir un docteur complet ;
- **Donnée** : le docteur créé ;
- **Erreur** : 400 si info incorrecte.



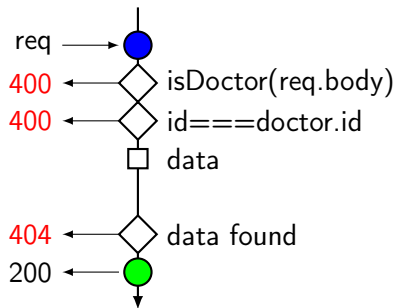
implémentation

```
1 import { isNewDoctor } from '../utils/guards';
2 doctorsController.post("/", (req: Request, res: Response) => {
3   const newDoctor: NewDoctorDTO = req.body; // extract the new doctor from
         the body
4   if (!isNewDoctor(newDoctor)) {
5     res.status(400).send("Invalid doctor");
6     return;
7   }
8   const newDoctor: NewDoctor = DoctorsMapper.fromNewDTO(newDoctor);
9   const doctor: Doctor = {
10     id: doctors.length + 1,
11     firstName: newDoctor.firstName,
12     lastName: newDoctor.lastName,
13     speciality : newDoctor.speciality
14   }
15   doctors.push(doctor); // adds the new doctor to the static list
16   res.status(201).send(DoctorsMapper.toDTO(doctor));
17 });
```



Objectif : modifier un docteur sur base d'information.

exemple : PUT `http://localhost:3000/doctors/:id`



- **Validation** : le body doit contenir un docteur complet et l'id doit être présent en paramètre et l'id passé en paramètre doit correspondre à l'id du docteur à modifier ;
- **Donnée** : le docteur modifié ;
- **Erreur** : 400 si info incorrecte, 404 si le docteur n'existe pas.



```
1 doctorsController.put("/:id", (req: Request, res: Response) => {
2   const updatedDoctorDTO: DoctorDTO = req.body;
3   if (!isDoctor(updatedDoctorDTO)) {
4     return res.status(400).send("Invalid doctor");
5   }
6   const updatedDoctor: Doctor = DoctorsMapper.fromDTO(updatedDoctorDTO);
7   let doctorIndex = -1;
8   for (let i = 0; i < doctors.length; i++) {
9     if (doctors[i].id === updatedDoctor.id) {
10      doctorIndex = i;
11      break;
12    }
13  }
14  if (doctorIndex === -1) {
15    return res.status(404).send("Doctor not found");
16  }
17  doctors[doctorIndex] = updatedDoctor;
18  res.status(200).send(DoctorsMapper.toDTO(updatedDoctor));
19 });
```




Si l'on souhaite mettre à jour uniquement l'un des champs d'un modèle, il est convenu d'utiliser la méthode PATCH. Cette méthode est généralement réservée à la modification d'un champ très précis.

Par exemple, si l'on a un modèle `Appointment` qui possède un état `status` qui peut prendre les valeurs `PENDING`, `CONFIRMED` et `CANCELLED`. Comme le changement d'un status d'un rendez-vous nécessite sans doute des vérifications, une route PATCH est mise en place et ressemblera à `http://localhost:3000/appointments/:id/status/:status`.

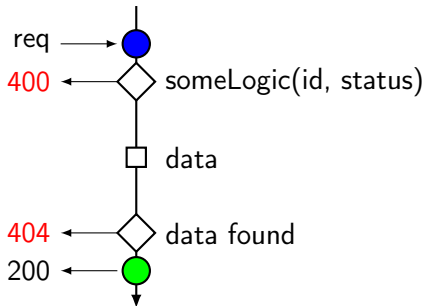


Le happy path d'une route utilisant la méthode PATCH est similaire à celui d'une route avec la méthode PUT. Toutefois, les routes PATCH sont généralement très dépendantes du contexte de l'application et il est difficile de donner une règle générale.



Exemple : PATCH

`http://localhost:3000/appointments/1/status/CONFIRMED`



- **Validation** : des validations dépendantes du contexte de l'application ;
- **Donnée** : le docteur modifié ;
- **Erreur** : 400 si info incorrecte, 404 si le docteur n'existe pas.

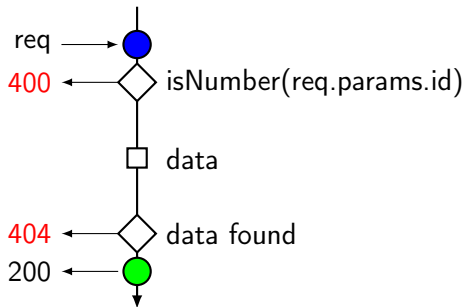
DELETE

happy path



Objectif : Supprimer un docteur sur base de son ID.

exemple : DELETE <http://localhost:3000/doctors/1>



- **Validation** : l'URL doit contenir un id de type nombre ;
- **Donnée** : aucune donnée n'est renvoyée ;
- **Erreur** : 400 si info incorrecte, 404 si le docteur n'existe pas.

DELETE



implémentation

```
1 import { isNumber } from '../utils/guards';
2 doctorsController.delete("/:id", (req: Request, res: Response) => {
3   const id = Number(req.params.id); // get the id from the ULR
4   if (!isNumber(id)) {
5     res.status(400).send("Invalid or missing id");
6     return;
7   }
8   let doctorIndex = -1;
9   for (let i = 0; i < doctors.length; i++) {
10     if (doctors[i].id === id) {
11       doctorIndex = i;
12       break;
13     }
14   }
15   if (doctorIndex === -1) {
16     res.status(404).send("Doctor not found");
17     return;
18   }
19   doctors.splice(doctorIndex, 1);
20   res.status(200).send();
```



6. Conclusion

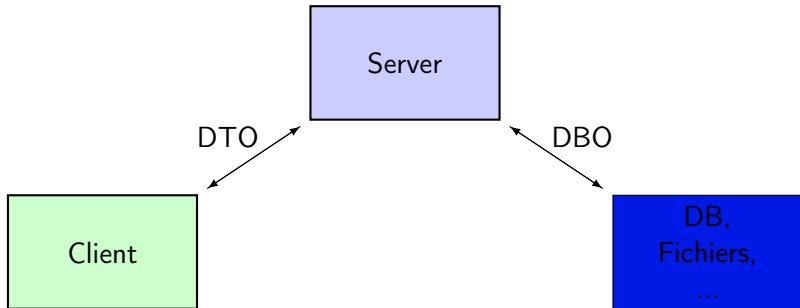


- 4 méthodes HTTP pour le CRUD (CREATE, READ, UPDATE, DELETE) ;
- 5 méthodes pour le contrôleur (GetAll, GetById, Create, Update, Delete) ;
- 5 Happy Path avec des validations et des erreurs ;



Conclusion

différentes représentations



Mapper

Les mappers permettent de transformer des objets entre différents formats. Ces classes permettent de centraliser la logique de transformation entre différents formats.

Conclusion

Les méthodes de guards



Les guards permettent de valider les données. Elles sont utilisées pour vérifier que les données sont présentes et de bons types.

- Toutes les fonctions commencent par `is`;
- Vérification qu'une donnée est un nombre : `isNumber`;
- Vérification qu'une donnée est un docteur : `isDoctor`;
- Vérification qu'une donnée est un patient : `isPatient`.

Elles sont toujours appelées en début de traitement de la requête dans le contrôleur. Si une donnée n'est pas valide, une réponse avec le status 400 est renvoyée.

Conclusion

Vérification du succès des opérations



Lorsqu'on effectue une opération sur des données d'une certaine ressource, il est important de vérifier le résultat de cette opération. Des erreurs peuvent-être rencontrées, comme par exemple un `id` non présent (opération GetById, Delete et Update).

Plus de scénarios seront étudiés dans les prochaines semaines.